

1/ Requête: La demande d'acquisition d'une ressource partagée R
{ Si R est disponible alors acquisition
sinon Attente du thread

2/ utilisation de la Ressource partagée

3/ Libération: Chaque ressource acquise doit être libérée

Deadlock (Interblocage)
• Dans des conditions normales de fonctionnement, un processus/thread ne peut utiliser une ressource qu'en suivant la séquence suivante :

Requête - Utilisation - Libération

Graphes d'allocation des ressources et Notations :

$P_i \rightarrow R_j$ } Le processus/thread
 $T_i \rightarrow R_j$ } demande la ressource partagée
Arc de requête

Deadlock (Interblocage)

$R_j \rightarrow P_i$ } La ressource est allouée
 $R_j \rightarrow T_i$ } aux processus/threads
lien d'allocation

- Dans des conditions normales de fonctionnement, un processus/thread ne peut utiliser une ressource qu'en suivant la séquence suivante :

On modélise les processus/threads par des cercles : $\textcircled{P}$ $\textcircled{T}$

Exemple :

Ressource partagée par $\square$, son instance par des points



$$P_i = \{P_1, P_2, P_3\}$$

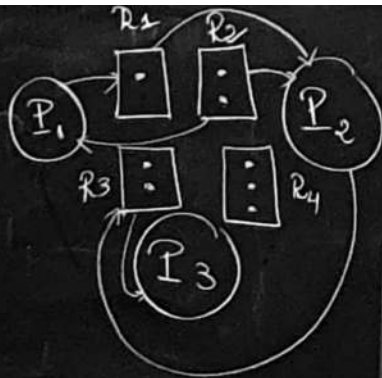
$$R_j = \{R_1, R_2, R_3, R_4\}$$

$$A = \{P_1 \rightarrow R_1, P_2 \rightarrow R_2, R_1 \rightarrow P_2, R_2 \rightarrow P_3, R_3 \rightarrow P_1, R_3 \rightarrow P_3\}$$

Deadlock (Interblocage)

- Dans des conditions normales de fonctionnement, un processus/thread ne peut utiliser une ressource qu'en suivant la séquence suivante :

Requête - Utilisation - Libération



→ : Arc de requête
= demande
de ressource

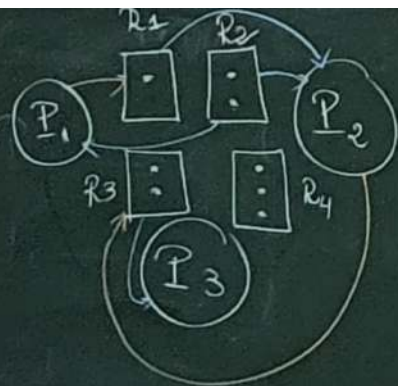
→ : Arc d'allocation
Acquisition des
ressources partagées

Deadlock (Interblocage)

- Dans des conditions normales de fonctionnement, un processus/thread ne peut utiliser une ressource qu'en suivant la séquence suivante :

type de ressource	Instance
R1	1
R2	2
R3	2
R4	3

Requête - Utilisation - Libération



→ : Arc de requête
= demande
de ressource

→ : Arc d'allocation
= Aquisition des
ressources partagées

type de ressource	Instance
R_1	1
R_2	2
R_3	2
R_4	3

Exemple 2: Robo Pathfinder

On considère 3 processus :

P1: pour le pilotage manuel qui reçoit les ondes wifi et opère avec les moteurs

P2: envoie le flux des vidéos via la liaison wifi

P3: autotest matériel

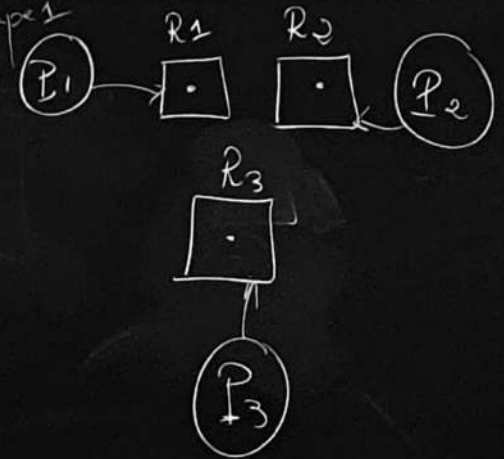
les 3 processus effectuent ces tâches simultanément.

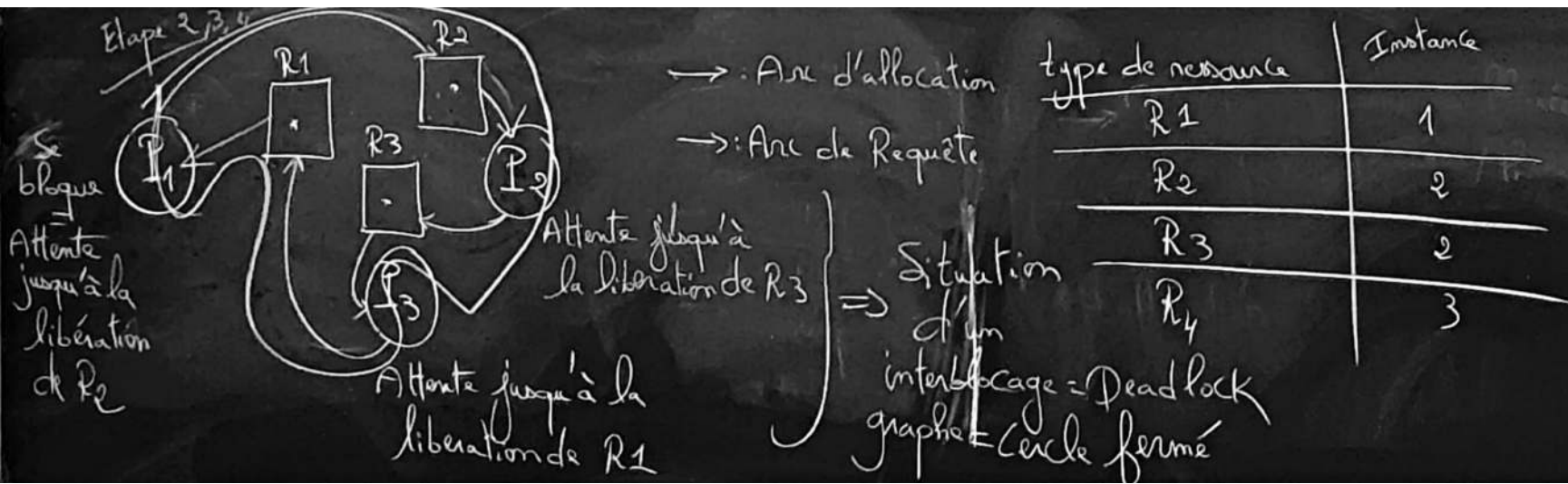
Etape 1

Etape 2

P1: pilotage manuel	P2: envoi du flux vidéo	P3: autotest matériel
Demandes R1 (moteurs)	Demandes R2 (wifi)	Demandes R3 (caméra)
Aquest R1	Aquest R2	Aquest R3
Demandes R2	Demandes R3	Demandes R1
Libères R1	Libères R2	Libères R3
Libères R2	Libères R3	Libères R1

Etape 1





Les sémaphores : Pour gérer les problèmes liés à la concurrence et imposer une synchronisation entre les processus/threads $\Rightarrow$ Indiquer la disponibilité des ressources

- Sémaphore binaire: 0 ou 1 : mutex

- Sémaphore variables: les valeurs varient jusqu'à N valeurs

On utilise 2 fonctions principales: $P(S)$: Proberen $\Rightarrow$ tester la disponibilité de la ressource (Puis-je utiliser cette ressource)
 $V(S)$: Verhogen $\Rightarrow$ libérer la ressource (Incrémenter S)
 $V(S) = S + 1$

$S \xrightarrow{\text{0 ou 1 (disponible)}} P(S) = S - 1$
 $\searrow 1 \text{ (disponible)}$

$$\boxed{\begin{array}{l} S > 0 \\ S \leq 0 \end{array}}$$

Exemple: Producteurs/Consommateurs

- Threads Producteurs: threads qui produisent des données (remplir tableaux, ...)
- Threads Consommateurs: threads qui consomment les données.
- Ressource partagée: Buffer



1/ Situation de blocage pour les threads producteurs: buffer plein
⇒ Les threads ne peuvent plus produire des données

2/ Situation de blocage pour threads consommateurs: buffer vide
⇒ Les threads ne peuvent consommer des données

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#define N 10
pthread_mutex_t mutex; // impose un mutex
                        // pour notre buffer
sem_t full;
sem_t empty;
```

```
pthread_mutex_init(&mutex);
sem_init(&empty, 0, N); // buffer vide
sem_init(&full, 0, 0); // buffer vide
```

```
void *producer(void *arg){
```

```
    int item;
```

```
    while (true){
```

```
        item = produce(item);
```

```
        sem_wait(&empty); // équivalent à  $P(S)$  = tester
```

```
        pthread_mutex_lock(&mutex); // si il y a des cases vides
```

```
        item = put(item);
```

```
        pthread_mutex_unlock(&mutex);
```

```
        sem_post(&full); //  $V(S)$  Valuer qu'il y a des cases remplies
```

Verrouillage du buffer

```
void *consumer(void *arg){
```

```
    int item;
```

```
    while (true){
```

```
        sem_wait(&full); //  $P(S)$  = Est ce qu'il y a une/des
```

(case(s) remplies)

```
        pthread_mutex_lock(&mutex);
```

```
        item = remove(item);
```

```
        pthread_mutex_unlock(&mutex);
```

```
        sem_post(&empty); // Indiquer qu'il y a des cases vides
```

Section critique